



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

ОТЧЕТ

по лабораторной работе №1

по курсу «Вычисления на GPU»

на тему: «Исследование производительности CPU и подсистемы памяти»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Лукашов И. В., Старший преподаватель
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ РАБОТЫ	4
1 Определение характеристик системы и подготовка средств профилирования	5
1.1 Конфигурация системы	5
1.2 Проверка <code>perf</code> и доступа к аппаратным счётчикам	5
1.3 Доступные аппаратные события	6
1.4 Вывод	7
2 Сборка тестового приложения и базовое профилирование	8
2.1 Параметры исследования	8
2.2 Измеренные показатели	8
2.3 Вывод	8
3 Исследование повторяемости измерений	9
3.1 Параметры исследования	9
3.2 Измеренные показатели	9
3.3 Вывод	9
4 Исследование размера рабочего набора и кэш-памяти	11
4.1 Параметры исследования	11
4.2 Измеренные показатели	12
4.3 Вывод	14
5 Исследование <code>stride</code> и пространственной локальности	15
5.1 Параметры исследования	15
5.2 Измеренные показатели	16
5.3 Вывод	16
6 Исследование влияния типа данных	18
6.1 Параметры исследования	18
6.2 Измеренные показатели	19
6.3 Вывод	19

7	Исследование TLB	21
7.1	Параметры исследования	21
7.2	Измеренные показатели	22
7.3	Вывод	23
8	Исследование предсказателя переходов	24
8.1	Параметры исследования	24
8.2	Измеренные показатели	25
8.3	Вывод	25
9	Анализ результатов	26
9.1	Выводы	27

ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы — изучить методы экспериментального исследования производительности центрального процессора и получить практические навыки анализа влияния подсистемы памяти и особенностей микроархитектуры CPU на выполнение программ.

Для достижения цели были поставлены и решены следующие задачи:

- 1) определить характеристики используемой вычислительной системы и подготовить средства аппаратного профилирования;
- 2) собрать и запустить тестовое приложение, выполнить базовое профилирование и определить основные показатели производительности CPU: время выполнения, `cycles`, `instructions` и IPC;
- 3) исследовать повторяемость измерений производительности и оценить разброс результатов;
- 4) исследовать влияние размера рабочего набора и поведения кэш-памяти на производительность программы;
- 5) исследовать влияние шага доступа к памяти `stride` и пространственной локальности обращений;
- 6) исследовать влияние типа данных на размер рабочего набора и характеристики выполнения программы;
- 7) исследовать работу TLB и влияние обращений к различным страницам памяти;
- 8) исследовать влияние предсказуемых и непредсказуемых условных переходов на производительность;
- 9) проанализировать полученные результаты и связать наблюдаемые эффекты с характеристиками используемой микроархитектуры.

Работа выполнялась на Apple M3 Pro под macOS, где `perf` недоступен (раздел 1.2).

1 Определение характеристик системы и подготовка средств профилирования

1.1 Конфигурация системы

```
system_profiler SPHardwareDataType
sysctl hw.physicalcpu hw.logicalcpu
sysctl hw.perflevel0 hw.perflevel1
getconf PAGESIZE
uname -r
sw_vers
```

Таблица 1.1 – Конфигурация системы

Параметр	Значение
Модель CPU	Apple M3 Pro (MacBook Pro, Mac15,6)
Число ядер	11
Число потоков	11
L1d на ядро	P: 131072 B; E: 65536 B
L2	P: 16777216 B; E: 4194304 B
L3 / LLC	—
Размер страницы памяти	16384 B
ОС / ядро	macOS 26.6.1 / Darwin 25.6.0
Версия perf	—

1.2 Проверка perf и доступа к аппаратным счётчикам

```
which perf
/usr/bin/time -l echo test
```

Утилита `perf` и системный вызов `perf_event_open()` есть только в Linux. В macOS счётчики `cycles` и `instructions` выводит `/usr/bin/time -l`.

Остальные события PMU считываются утилитой `mperf` (файлы `mperf.c` и `mperf`), аналогом `perf stat -e`. Она подключает к программе библиотеку, которая через закрытый фреймворк `kperf` настраивает счётчики и выводит их значения для основного потока программы.

```
make libmperf.dylib
sudo ./mperf -e FIXED_CYCLES, FIXED_INSTRUCTIONS, INST_INT_LD, \
INST_SIMD_LD, L1D_CACHE_MISS_LD_NONSPEC ./daxpy 1000000 100 1
```

1.3 Доступные аппаратные события

```
python3 scripts/events.py > events.txt
grep -i cache events.txt
grep -i tlb events.txt
grep -i branch events.txt
```

Листинг 1.1 – Вывод списка событий PMU

```
import plistlib

with open("/usr/share/kpep/as3.plist", "rb") as f:
    events = plistlib.load(f)["system"]["cpu"]["events"]
for name, event in sorted(events.items()):
    print(name.ljust(36), event.get("description", ""))
```

Из вывода `grep` исключены события, в описании которых кэш или переходы только упоминаются (`ATOMIC_OR_EXCLUSIVE_*`, `INST_INT_ST`, `INST_LDST`, `INST_INT_ALU`, `FETCH_RESTART`, `FLUSH_RESTART_OTHER_NONSPEC`).

Таблица 1.2 – Доступные события кэша

Событие	Описание
L1D_CACHE_MISS_LD	загрузки с промахом в L1d
L1D_CACHE_MISS_LD_NONSPEC	завершённые загрузки с промахом в L1d
L1D_CACHE_MISS_ST	сохранения с промахом в L1d
L1D_CACHE_MISS_ST_NONSPEC	завершённые сохранения с промахом в L1d
L1D_CACHE_WRITEBACK	изменённые строки, записанные из L1d в общий L2
L1I_CACHE_MISS_DEMAND	промахи L1i при выборке инструкций, требующие загрузки новой строки
LDST_UNIT_OLD_L1D_CACHE_MISS	такты, в течение которых старая операция загрузки или сохранения ожидает данные после промаха L1d
LDST_UNIT_WAITING_OLD_L1D_CACHE_MISS	то же, при условии что планировщик не выдал ни одной микрооперации
MAP_DISPATCH_BUBBLE_IC	такты простоя блока Map из-за L1i

Таблица 1.3 – Доступные события TLB

Событие	Описание
L1D_TLB_ACCESS	обращения загрузок и сохранений к L1 dTLB
L1D_TLB_MISS	обращения загрузок и сохранений с промахом в L1 dTLB
L1D_TLB_MISS_NONSPEC	завершённые загрузки и сохранения с промахом в L1 dTLB
L1D_TLB_FILL	трансляции, загруженные в L1 dTLB
L2_TLB_MISS_DATA	загрузки и сохранения с промахом в L2 TLB
L1I_TLB_MISS_DEMAND	выборки инструкций с промахом в L1 iTLB
L1I_TLB_FILL	трансляции, загруженные в L1 iTLB
L2_TLB_MISS_INSTRUCTION	выборки инструкций с промахом в L2 TLB
MAP_DISPATCH_BUBBLE_ITLB	такты простоя блока Map из-за L1 iTLB

Таблица 1.4 – Доступные события переходов

Событие	Описание
INST_BRANCH	завершённые инструкции перехода, включая вызовы и возвраты
INST_BRANCH_TAKEN	завершённые переходы, которые были выполнены (taken)
INST_BRANCH_CALL	завершённые вызовы подпрограмм
INST_BRANCH_RET	завершённые возвраты из подпрограмм
INST_BRANCH_INDIR	завершённые косвенные переходы, включая косвенные вызовы
BRANCH_MISPRED_NONSPEC	все неверно предсказанные переходы
BRANCH_COND_MISPRED_NONSPEC	неверно предсказанные условные переходы
BRANCH_INDIR_MISPRED_NONSPEC	неверно предсказанные косвенные переходы, включая вызовы и возвраты
BRANCH_CALL_INDIR_MISPRED_NONSPEC	неверно предсказанные косвенные вызовы
BRANCH_RET_INDIR_MISPRED_NONSPEC	неверно предсказанные возвраты из подпрограмм

Недоступные события: `cache-references`, `cache-misses`, `LLC-loads`, `LLC-load-misses`. В базе событий Apple M3 нет событий кэша L2 и SLC, события кэша доступны только для L1.

1.4 Вывод

В первом этапе получена и зафиксирована конфигурация системы.

2 Сборка тестового приложения и базовое профилирование

2.1 Параметры исследования

В качестве тестовой нагрузки используется вычислительный цикл:

$$y_i = ax_i + y_i.$$

Программа позволяет изменять количество элементов N , число повторов `repeats`, шаг доступа `stride` и тип данных.

```
g++ daxpy.cpp -O3 -std=c++17 -o daxpy
./daxpy 1000000 100 1
/usr/bin/time -l ./daxpy 1000000 100 1
```

Здесь 1000000 — количество элементов, 100 — число повторов, 1 — `stride`.

$$IPC = \frac{Instructions}{Cycles}.$$

2.2 Измеренные показатели

Таблица 2.1 – Базовые показатели (`./daxpy 1000000 100 1`)

Показатель	Значение
Время выполнения, с	0.04
<code>cycles</code>	125609823
<code>instructions</code>	721017564
IPC	5.7401

2.3 Вывод

На втором этапе получены корректно собранная и запускаемая программа, а также набор базовых метрик: время выполнения, `cycles`, `instructions` и рассчитанное значение IPC.

3 Исследование повторяемости измерений

3.1 Параметры исследования

Один и тот же запуск повторяется 10 раз, после чего замеряются статистики:

```
python3 scripts/repeatability.py
```

Листинг 3.1 – Многократный запуск и расчёт статистик

```
import re
import statistics as st
import subprocess
import time

args = ["/usr/bin/daxpy", "1000000", "100", "1"]
subprocess.run(args, capture_output=True)
t, cyc, ins = [], [], []
for _ in range(10):
    s = time.perf_counter()
    subprocess.run(args, capture_output=True)
    t.append(time.perf_counter() - s)
    err = subprocess.run(["/usr/bin/time", "-l", *args],
                        capture_output=True, text=True).stderr
    cyc.append(int(re.search(r"(\d+)\s+cycles elapsed", err)[1]))
    ins.append(int(re.search(r"(\d+)\s+instructions retired", err)[1]))
for name, v in [("time", t), ("cycles", cyc), ("instructions", ins)]:
    m, s = st.mean(v), st.stdev(v)
    print(f"{name:13} {m:.6g} +- {100 * s / m:.2f}%")
```

3.2 Измеренные показатели

Таблица 3.1 – Результаты многократного запуска ./daxpy 1000000 100 1

Показатель	Среднее значение	Разброс
Время выполнения, с	0.0340062	7.14%
cycles	1.29648e+08	8.06%
instructions	7.20968e+08	0.09%

3.3 Вывод

В третьем этапе получены среднее значение и оценка разброса времени/метрик. Измерения воспроизводимы, однако случаются выбросы в некото-

рых отдельных замерах.

4 Исследование размера рабочего набора и кэш-памяти

4.1 Параметры исследования

Для типа `double` и двух массивов приблизительный размер рабочего набора:

$$W = 2N \times 8.$$

При изменении N число повторов `repeats` корректируется так, чтобы величина $N \times repeats = 2^{30}$ оставалась постоянной. Нормализованная стоимость одного обращения и доля промахов L1d:

$$cycles/access = \frac{cycles}{N \times repeats},$$

$$L1d \text{ miss rate} = \frac{L1D_CACHE_MISS_LD_NONSPEC}{INST_INT_LD + INST_SIMD_LD}.$$

Событий кэша L2 в PMU Apple M3 нет. Каждый промах L1d при загрузке — обращение к L2, а удалённость источника данных оценивается по числу тактов ожидания на один промах:

$$\frac{LDST_UNIT_OLD_L1D_CACHE_MISS}{L1D_CACHE_MISS_LD_NONSPEC}.$$

```
make libmperf.dylib
sudo python3 scripts/working_set.py
```

Листинг 4.1 – Измерения для разных размеров рабочего набора

```
import re
import subprocess
import time

events = ",".join([
    "FIXED_CYCLES", "FIXED_INSTRUCTIONS", "INST_INT_LD", "INST_SIMD_LD",
    "L1D_CACHE_MISS_LD_NONSPEC", "LDST_UNIT_OLD_L1D_CACHE_MISS",
])

total = 2**30

subprocess.run(["./daxpy", "256", "1", "1"], capture_output=True)
print("W_KiB N repeats time_s cycles instructions IPC cycles_per_access",
      "loads l1d_misses l1d_miss_rate wait_cycles wait_per_miss")
for kib in [4, 16, 32, 64, 256, 1024, 4096, 16384, 65536]:
```

```

n = kib * 1024 // 16
rep = total // n
args = ["/daxpy", str(n), str(rep), "1"]
s = time.perf_counter()
subprocess.run(args, capture_output=True)
t = time.perf_counter() - s
err = subprocess.run(["./mperf", "-e", events, *args],
                    capture_output=True, text=True).stderr
c = {name: int(val) for val, name in re.findall(r"(\d+)\s+(\w+)", err)}
cyc, ins = c["FIXED_CYCLES"], c["FIXED_INSTRUCTIONS"]
loads = c["INST_INT_LD"] + c["INST_SIMD_LD"]
miss = c["L1D_CACHE_MISS_LD_NONSPEC"]
wait = c["LDST_UNIT_OLD_L1D_CACHE_MISS"]
print(kib, n, rep, f"{t:.3f}", cyc, ins, f"{ins / cyc:.2f}",
      f"{cyc / (n * rep):.3f}", loads, miss, f"{100 * miss / loads:.2f}",
      wait, f"{wait / miss:.2f}" if miss else "nan")

```

4.2 Измеренные показатели

Таблица 4.1 – Показатели производительности

W	N	repeats	Время, с	cycles	IPC	<i>cycles/</i> <i>access</i>
4 KiB	256	4194304	0.312	1244479032	6.05	1.159
16 KiB	1024	1048576	0.291	1148942656	6.55	1.070
32 KiB	2048	524288	0.286	1145517636	6.56	1.067
64 KiB	4096	262144	0.283	1123702031	6.69	1.047
256 KiB	16384	65536	0.295	1167057147	6.44	1.087
1 MiB	65536	16384	0.304	1174554774	6.40	1.094
4 MiB	262144	4096	0.316	1256712693	5.98	1.170
16 MiB	1048576	1024	0.319	1240140494	6.07	1.155
64 MiB	4194304	256	0.331	1288751981	5.86	1.200

Таблица 4.2 – Показатели промахов кэша

W	Загрузки	Прوماхи L1d	L1d miss rate, %	Такты ожидания	Тактов ожидания на промах
4 KiB	2147596310	2858	0.00	17128	5.99
16 KiB	2147596667	5104	0.00	29308	5.74
32 KiB	2147596904	5498	0.00	20939	3.81
64 KiB	2147597402	11723	0.00	32300	2.76
256 KiB	2147600785	23299225	1.08	4236049	0.18
1 MiB	2147613032	5889122	0.27	1157559	0.20
4 MiB	2147662114	59094045	2.75	17935829	0.30
16 MiB	2147858565	60548157	2.82	28326887	0.47
64 MiB	2148645014	59736871	2.78	25737795	0.43

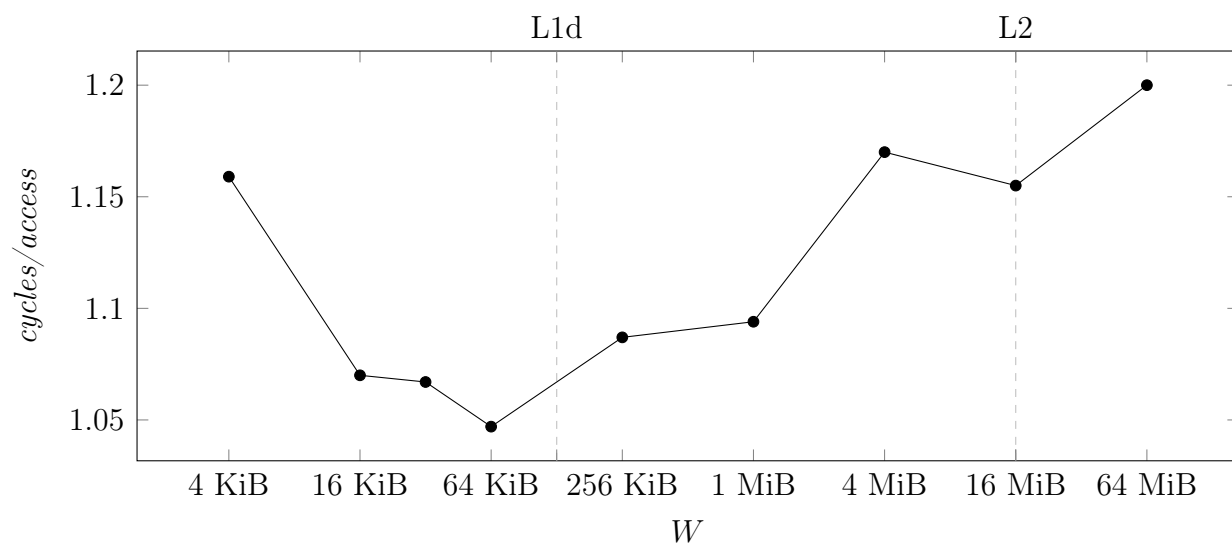


Рисунок 4.1 – Зависимость $cycles/access$ от размера рабочего набора

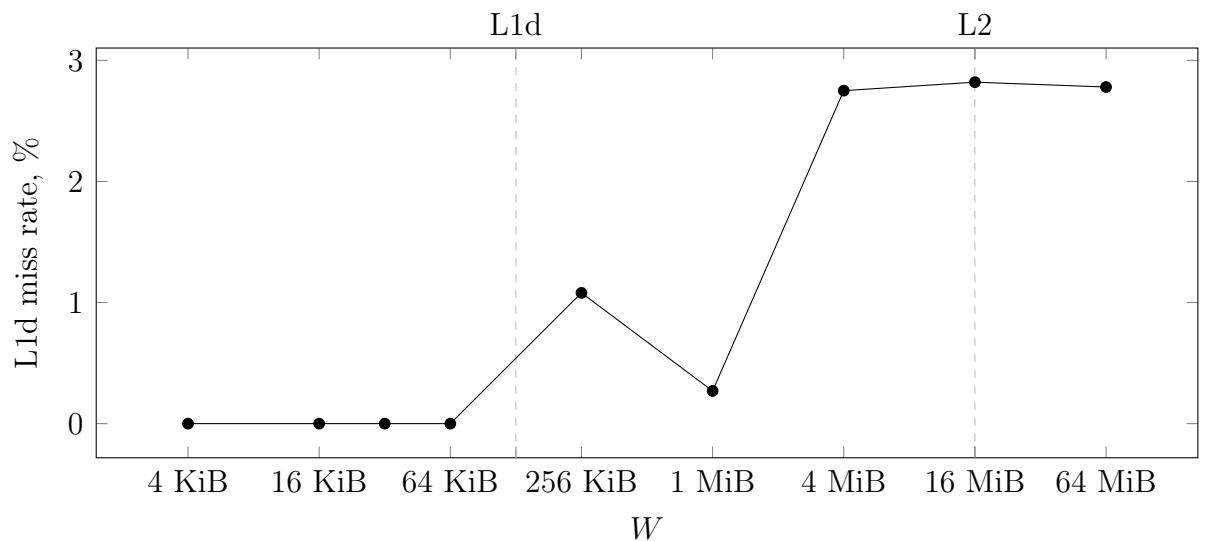


Рисунок 4.2 – Зависимость cache miss rate от размера рабочего набора

4.3 Вывод

В четвёртом этапе получены зависимости времени выполнения, cycles, IPC и показателей промахов кэша от размера рабочего набора. До 64 KiB рабочий набор помещается в L1d и промахов практически нет, после 128 KiB появляются промахи L1d, после 16 MiB растёт число тактов ожидания на промах, то есть данные идут уже не из L2. При этом cycles/access меняется слабо, так как аппаратная предвыборка скрывает большую часть задержек.

5 Исследование stride и пространственной локальности

5.1 Параметры исследования

Размер массива превышает объём исследуемого кэша: $N = 10^7$, $repeats = 20$. Размер строки кэша — 128 В.

Количество фактических обращений и нормализованная стоимость одного обращения:

$$N_{access} = \frac{N}{stride} \times repeats, \quad cycles/access = \frac{cycles}{N_{access}}.$$

L1d miss rate рассчитывается так же, как в главе 4. Утилита `mperf`, как и `perf stat`, учитывает весь процесс, включая выделение и заполнение массивов. Эта часть не зависит от `stride` и при больших `stride` сопоставима с самим циклом, поэтому из всех показателей вычитается запуск с $repeats = 0$.

```
sudo python3 scripts/stride.py
```

Листинг 5.1 – Измерения для разных значений `stride`

```
import re
import statistics
import subprocess

events = "FIXED_CYCLES,INST_INT_LD,INST_SIMD_LD,L1D_CACHE_MISS_LD_NONSPEC"
n, rep = 10**7, 20

def measure(repeats, stride):
    args = ["/daxpy", str(n), str(repeats), str(stride)]
    err = subprocess.run(["/mperf", "-e", events, *args],
                        capture_output=True, text=True).stderr
    c = {name: int(val) for val, name in re.findall(r"(\d+)\s+(\w+)", err)}
    loads = c["INST_INT_LD"] + c["INST_SIMD_LD"]
    return c["FIXED_CYCLES"], loads, c["L1D_CACHE_MISS_LD_NONSPEC"]

subprocess.run(["/daxpy", str(n), "0", "1"], capture_output=True)
base = [statistics.median(v) for v in zip(*(measure(0, 1) for _ in range(3)))]
print(f"# baseline (repeats=0): cycles={base[0]:.0f} loads={base[1]:.0f},
      f"l1d_misses={base[2]:.0f}")
print("stride N_access cycles cycles_per_access loads l1d_misses l1d_miss_rate")
for s in [1, 2, 4, 8, 16, 32, 64]:
    acc = n // s * rep
    cyc, loads, miss = (a - b for a, b in zip(measure(rep, s), base))
```

```
print(s, acc, round(cyc), f"{cyc / acc:.2f}", round(loads), round(miss),
      f"{100 * miss / loads:.2f}")
```

5.2 Измеренные показатели

Показатели холостого запуска: `cycles` — 64410983, загрузки — 2612952, промахи L1d — 204891.

Таблица 5.1 – Зависимость показателей от `stride`

<code>stride</code>	N_{access}	<code>cycles</code>	$cycles / access$	Прوماхи L1d	L1d miss rate, %
1	200000000	230063313	1.15	11096310	2.77
2	100000000	165006890	1.65	13896876	6.95
4	50000000	163639851	3.27	40323205	40.32
8	25000000	156800735	6.27	37317107	74.63
16	12500000	233745496	18.70	24352419	97.41
32	6250000	149043598	23.85	11925774	95.41
64	3125000	81764859	26.16	6106280	97.70

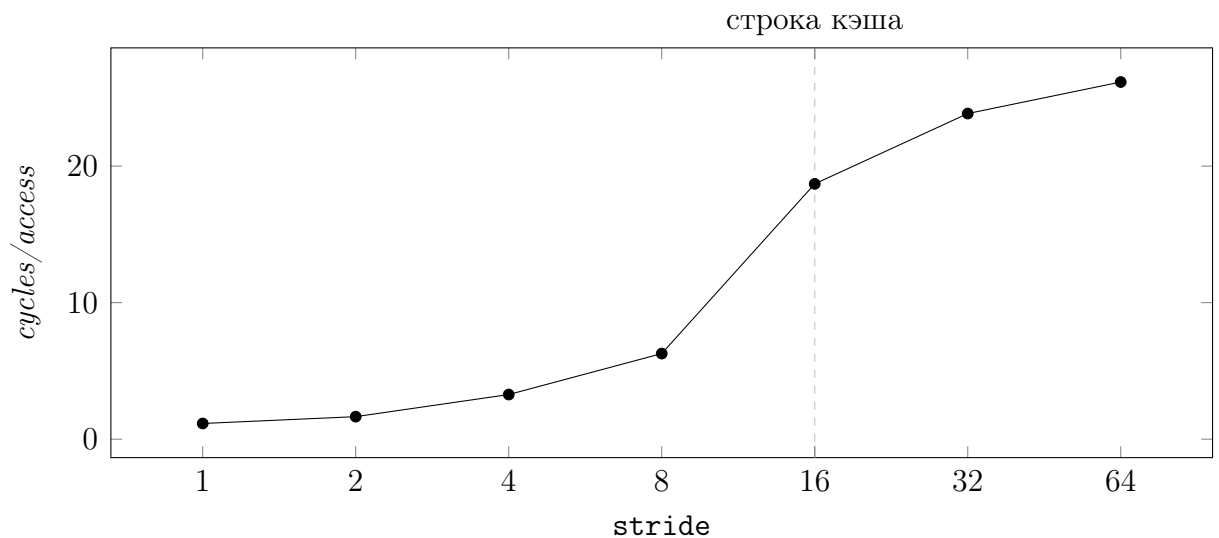


Рисунок 5.1 – Зависимость $cycles/access$ от `stride`

5.3 Вывод

В пятом этапе получена зависимость $cycles/access$ и показателей промахов кэша от `stride`. До `stride` = 8 стоимость обращения растёт почти вдвое при каждом увеличении шага, так как из загруженной строки кэша используется всё меньше элементов. При `stride` = 16 каждое обращение попадает в

новую строку ($128\text{ B} = 16$ элементов double), промахи L1d достигают 97% и cycles/access резко возрастает, дальше рост замедляется.

6 Исследование влияния типа данных

6.1 Параметры исследования

Одинаковые замеры выполняются для `run_daxpy<double>` и `run_daxpy<float>` с одинаковыми значениями `N`, `repeats` и `stride`:

$$W = 2N \times \text{sizeof}(T).$$

Тип элементов задаётся при сборке макросом `TYPE`. При $N = 4096$ рабочий набор обоих типов помещается в L1d, при $N = 12288$ и $N = 1572864$ рабочий набор `float` помещается в L1d и L2 соответственно, а `double` — нет. Как и в главе 4, $N \times \text{repeats} = 2^{30}$, $\text{stride} = 1$, а cycles/access и L1d miss rate рассчитываются по тем же формулам.

```
g++ daxpy.cpp -O3 -std=c++17 -o daxpy
g++ daxpy.cpp -O3 -std=c++17 -DTYPE=float -o daxpy_float
sudo python3 scripts/data_type.py
```

Листинг 6.1 – Измерения для `float` и `double`

```
import re
import subprocess
import time

events = ",".join([
    "FIXED_CYCLES", "INST_INT_LD", "INST_SIMD_LD",
    "L1D_CACHE_MISS_LD_NONSPEC", "LDST_UNIT_OLD_L1D_CACHE_MISS",
])

total = 2**30
types = [("./daxpy_float", "float", 4), (("./daxpy", "double", 8)]

print("N repeats type W_KiB time_s cycles cycles_per_access",
      "loads l1d_misses l1d_miss_rate wait_per_miss")
for n in [4096, 12288, 1572864]:
    rep = total // n
    for binary, name, size in types:
        args = [binary, str(n), str(rep), "1"]
        subprocess.run(args, capture_output=True)
        s = time.perf_counter()
        subprocess.run(args, capture_output=True)
        t = time.perf_counter() - s
        err = subprocess.run(["./mperf", "-e", events, *args],
                              capture_output=True, text=True).stderr
        c = {k: int(v) for v, k in re.findall(r"(\d+)\s+(\w+)", err)}
```

```

cyc, loads = c["FIXED_CYCLES"], c["INST_INT_LD"] + c["INST_SIMD_LD"]
miss = c["L1D_CACHE_MISS_LD_NONSPEC"]
wait = c["LDST_UNIT_OLD_L1D_CACHE_MISS"]
print(n, rep, name, 2 * n * size // 1024, f"{t:.3f}", cyc,
      f"{cyc / (n * rep):.3f}", loads, miss, f"{100 * miss / loads:.2f}",
      f"{wait / miss:.2f}" if miss else "nan")

```

6.2 Измеренные показатели

Таблица 6.1 – Показатели производительности float и double

N	repeats	Тип	W	Время, с	cycles	<i>cycles / access</i>
4096	262144	float	32 KiB	0.286	1142626701	1.064
4096	262144	double	64 KiB	0.299	1156977504	1.078
12288	87381	float	96 KiB	0.287	1099099413	1.024
12288	87381	double	192 KiB	0.307	1168321217	1.088
1572864	682	float	12 MiB	0.293	1197144128	1.116
1572864	682	double	24 MiB	0.313	1230434748	1.147

Таблица 6.2 – Показатели промахов кэша float и double

N	Тип	Прوماхи L1d	L1d miss rate, %	Тактов ожидания на промах
4096	float	8964	0.00	2.98
4096	double	15788	0.00	2.99
12288	float	115395	0.01	1.10
12288	double	30539590	1.42	0.17
1572864	float	49020204	2.28	0.27
1572864	double	59025652	2.75	0.41

6.3 Вывод

В шестом этапе получено сравнение float и double при одинаковых N , repeats и stride. Элемент float вдвое меньше, поэтому при том же N рабочий набор float вдвое меньше и помещается в более близкий уровень кэша: при $N = 12288$ рабочий набор float (96 KiB) помещается в L1d и промахов практически нет, а у double (192 KiB) доля промахов L1d составляет 1,42%. При $N = 1572864$ рабочий набор float (12 MiB) помещается в L2, а double (24 MiB) — нет, поэтому у double больше тактов ожидания на промах (0,41 против

0,27). Когда оба рабочих набора помещаются в L1d, cycles/access почти не отличается, так как число инструкций на элемент одинаково.

7 Исследование TLB

7.1 Параметры исследования

Для страницы 16384 В и типа `double` на одной странице размещается

$$16384/8 = 2048$$

элементов, поэтому $stride = 2048$ используется для обращения приблизительно к одному элементу на страницу.

Число затрагиваемых страниц обоих массивов и количество обращений:

$$pages = \frac{2N \times 8}{16384}, \quad N_{access} = \frac{N}{stride} \times repeats = 2^{24}.$$

Одно обращение — одна итерация цикла, затрагивающая страницы обоих массивов.

Доступные TLB-события приведены в таблице 1.3. Аналоги `dTLB-loads` и `dTLB-load-misses` — `L1D_TLB_ACCESS` и `L1D_TLB_MISS`, дополнительно считается `L2_TLB_MISS_DATA`:

$$dTLB \text{ miss rate} = \frac{L1D_TLB_MISS}{L1D_TLB_ACCESS}.$$

Как и в главе 5, из всех показателей вычитается запуск с $repeats = 0$, который выполняется для каждого N .

```
sudo python3 scripts/tlb.py
```

Листинг 7.1 – Измерения для разного числа страниц

```
import re
import statistics
import subprocess

events = "FIXED_CYCLES,L1D_TLB_ACCESS,L1D_TLB_MISS,L2_TLB_MISS_DATA"
total = 2**24

def measure(n, repeats):
    args = ["/daxpy", str(n), str(repeats), "2048"]
    err = subprocess.run(["./mperf", "-e", events, *args],
                        capture_output=True, text=True).stderr
    c = {k: int(v) for v, k in re.findall(r"(\d+)\s+(\w+)", err)}
    return [c[e] for e in events.split(",")]
```

```

subprocess.run(["./daxpy", "1024", "1", "2048"], capture_output=True)
print("pages N repeats N_access cycles cycles_per_access",
      "l1_dtlb_accesses l1_dtlb_misses l1_dtlb_miss_rate l2_tlb_misses")
for k in range(17, 26):
    n = 2**k
    rep = total // (n // 2048)
    base = [statistics.median(v) for v in zip(*(measure(n, 0) for _ in range(3)))]
    cyc, acc_tlb, miss1, miss2 = (a - b for a, b in zip(measure(n, rep), base))
    acc = n // 2048 * rep
    print(n // 1024, n, rep, acc, round(cyc), f"{cyc / acc:.2f}", round(acc_tlb),
          round(miss1), f"{100 * miss1 / acc_tlb:.2f}", round(miss2))

```

7.2 Измеренные показатели

Таблица 7.1 – Зависимость производительности от числа затрагиваемых страниц

Страниц	N	repeats	cycles	$cycles/access$
128	131072	262144	357716639	21.32
256	262144	131072	444587464	26.50
512	524288	65536	423191687	25.22
1024	1048576	32768	430167662	25.64
2048	2097152	16384	779460023	46.46
4096	4194304	8192	692875488	41.30
8192	8388608	4096	585662352	34.91
16384	16777216	2048	486510270	29.00
32768	33554432	1024	521253294	31.07

Таблица 7.2 – Зависимость TLB misses от числа затрагиваемых страниц

Страниц	Обращения к L1 dTLB	Промехи L1 dTLB	dTLB miss rate, %	Промехи L2 TLB
128	50333787	1152	0.00	16
256	92306340	41968097	45.47	400
512	93324298	42989568	46.06	42
1024	96310823	45964302	47.72	1769
2048	110235436	59849233	54.29	24631
4096	156936737	106530159	67.88	33583663
8192	373205322	322676315	86.46	33608732
16384	564996073	514270752	91.02	33680098
32768	779877073	728399183	93.40	33983761

7.3 Вывод

В седьмом этапе получена зависимость количества и доли TLB misses и производительности от числа используемых страниц. Промахи L1 dTLB появляются при переходе от 128 к 256 страницам, а промахи L2 TLB — при переходе от 2048 к 4096 страницам, после чего промахивается практически каждое обращение к странице. После переполнения L1 dTLB cycles/access растёт с 21 до 26 тактов, однако наибольшее значение (46 тактов) наблюдается при 2048 страницах, ещё до переполнения L2 TLB, поэтому производительность в этом исследовании определяется не только TLB.

8 Исследование предсказателя переходов

8.1 Параметры исследования

Программа `branch_test` заполняет массив условий `cond` и массив данных `data` из N элементов и выполняет тестовый цикл:

```
for (size_t r = 0; r < repeats; r++) {  
    for (size_t i = 0; i < n; i++) {  
        if (cond[i]) {  
            sum += data[i];  
        }  
    }  
}
```

Режимы:

- предсказуемый — условие имеет почти постоянное поведение: истинно для первой половины массива и ложно для второй;
- непредсказуемый — используется заранее сформированная случайная последовательность условий.

В обоих режимах условие истинно примерно для половины элементов, поэтому объём вычислений одинаков.

Аналоги `branches` и `branch-misses` (таблица 1.4) — `INST_BRANCH` и `BRANCH_MISPRED_NONSPEC`. Доля ошибочных предсказаний:

$$\text{Branch miss rate} = \frac{\text{branch misses}}{\text{branches}}.$$

Как и в главе 5, из всех показателей вычитается запуск с `repeats = 0`.

```
g++ branch_test.cpp -O3 -std=c++17 -o branch_test  
sudo python3 scripts/branch.py
```

Листинг 8.1 – Измерения для двух режимов условия

```
import re  
import statistics  
import subprocess  
import time  
  
events = "FIXED_CYCLES, INST_BRANCH, BRANCH_MISPRED_NONSPEC"  
n, rep = 10**6, 100
```

```

def measure(mode, repeats):
    args = ["/branch_test", str(mode), str(n), str(repeats)]
    times = []
    for _ in range(3):
        s = time.perf_counter()
        subprocess.run(args, capture_output=True)
        times.append(time.perf_counter() - s)
    err = subprocess.run(["./mperf", "-e", events, *args],
                        capture_output=True, text=True).stderr
    c = {k: int(v) for v, k in re.findall(r"(\d+)\s+(\w+)", err)}
    return [statistics.median(times)] + [c[e] for e in events.split(",")]

subprocess.run(["./branch_test", "0", "1000", "1"], capture_output=True)
print("mode time_s cycles branches branch_misses branch_miss_rate")
for mode, name in [(0, "predictable"), (1, "random")]:
    t, cys, br, miss = (a - b for a, b in zip(measure(mode, rep), measure(mode, 0)))
    print(name, f"{t:.3f}", cys, br, miss, f"{100 * miss / br:.2f}")

```

8.2 Измеренные показатели

Таблица 8.1 – Предсказуемые и непредсказуемые переходы

Режим	branches	branch-misses	Branch miss rate, %	Время, с	cycles
Предсказуемый	118750413	801	0.00	0.030	106547396
Непредсказуемый	118762321	47855817	40.30	0.223	913378231

8.3 Вывод

В восьмом этапе получены значения branches, branch-misses, branch miss rate и времени выполнения для предсказуемого и непредсказуемого режимов. При одинаковом числе переходов в непредсказуемом режиме доля ошибочных предсказаний составляет 40,3% против практически нулевой в предсказуемом, а число тактов увеличивается в 8.6 раза.

9 Анализ результатов

В таблице 9.1 результаты этапов 4–8 сопоставлены с характеристиками системы из таблицы 1.1. Так как количество фактических операций в разных режимах различается, для сравнения используются нормализованные метрики: cycles/access, L1d miss rate, число тактов ожидания на промах и branch miss rate.

Таблица 9.1 – Связь наблюдаемых изменений с характеристиками системы

Причина	Этап	Характеристика системы	Наблюдаемое изменение
Выход рабочего набора за пределы уровней кэша	4	L1d 128 KiB, L2 16 MiB	До 64 KiB промахов L1d нет, после 128 KiB появляются промахи L1d (до 2,8%), после 16 MiB число тактов ожидания на промах растёт с 0,2–0,3 до 0,43–0,47; cycles/access растёт лишь с 1,05 до 1,20
Ухудшение пространственной локальности	5	Строка кэша 128 B	cycles/access растёт с 1,15 при stride = 1 до 18,7 при stride = 16, когда каждое обращение попадает в новую строку (97% промахов L1d)
Изменение размера элементов	6	L1d 128 KiB, L2 16 MiB	При N = 12288 рабочий набор float (96 KiB) помещается в L1d, а double (192 KiB) — нет: 0,01% и 1,42% промахов L1d; при N = 1572864 тактов ожидания на промах 0,27 у float и 0,41 у double
Увеличение числа TLB misses	7	Страница 16 KiB	Промахи L1 dTLB появляются при переходе от 128 к 256 страницам (cycles/access растёт с 21 до 26), промахи L2 TLB — при переходе от 2048 к 4096 страницам
Увеличение числа ошибок предсказания переходов	8	—	Branch miss rate 40,3% против практически нулевой, число тактов больше в 8,6 раза, около 17 тактов на одну ошибку

9.1 Выводы

В результате работы получены аппаратные показатели производительности Apple M3 Pro для последовательного и разреженного доступа к данным и для условных переходов с разной предсказуемостью. Наибольшее влияние на время выполнения оказывают пространственная локальность и ошибки предсказания переходов. Выход рабочего набора за пределы L1d и L2 при последовательном доступе увеличивает cycles/access лишь примерно на 15%, так как аппаратная предвыборка заранее загружает строки кэша.